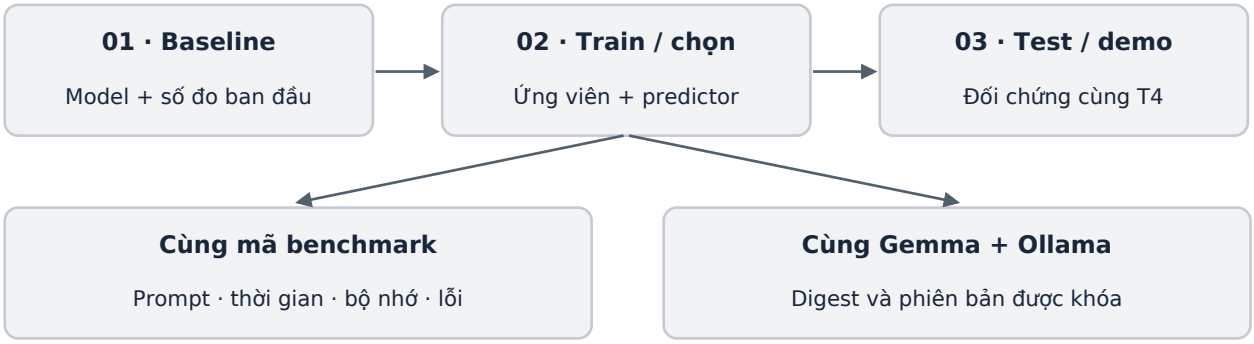
GEMMA 4 12B TRÊN COLAB T4

Hiểu từ đầu, thiết kế hệ thống và kiểm chứng tối ưu

Cẩm nang thực hành cho đồ án Decoding LLMs trong điều kiện giới hạn phần cứng



Bám vào notebook `Ollama+Gemma4_12B(1).ipynb` bạn đã gửi. Mô hình giữ nguyên là `gemma4:12b`, engine là Ollama, thiết bị mục tiêu là một GPU NVIDIA T4 trong Google Colab.

Ba notebook đi theo một vòng thực nghiệm: **đo ban đầu → thu dữ liệu và train bộ chọn cấu hình → đánh giá hệ thống trên dữ liệu chưa dùng để chọn cấu hình**.

Tài liệu này giải thích cả ý tưởng và những gì mã nguồn thực sự làm. Các phép tính số học minh họa không phải kết quả benchmark T4. Gói bàn giao chưa có phiên chạy Gemma thật trên T4 trong môi trường soạn tài liệu.

Cách đọc: đọc phần 01-07 để hiểu nền tảng; phần 08-15 để hiểu thiết kế và tối ưu; phần 16-22 để chạy và đánh giá; các phần cuối dùng khi viết báo cáo, sửa lỗi và bảo vệ đồ án.

Điểm phải nhớ ngay: Gemma sinh câu trả lời; Ollama vận hành Gemma; Colab cung cấp máy tính; bộ tối ưu của nhóm đo và chọn cách vận hành. Đây là bốn thành phần khác nhau.

Phiên bản tài liệu: 13/10/2026. Nguồn kỹ thuật được đối chiếu với tài liệu chính thức; các tag và hành vi engine có thể thay đổi, nên notebook ghi phiên bản cùng mã định danh nội dung.

01. Bạn đang xây dựng sản phẩm gì?

Hãy hình dung bạn có một trợ lý học tập. Bạn hỏi: “Giải thích vòng lặp for bằng ví dụ quản lý sinh viên”. Hệ thống nhận câu hỏi, đưa vào Gemma và hiển thị từng phần câu trả lời. Đồ án của bạn nghiên cứu cách làm cho quá trình đó phù hợp hơn với bộ nhớ và thời gian chờ trên T4.

Bạn không bắt đầu bằng việc tạo một LLM từ số không. Bạn dùng model có sẵn và xây phần mềm thực nghiệm quanh nó: khởi động engine, kiểm tra tài nguyên, gửi yêu cầu, thu số đo, thử cấu hình, học từ số đo và vận hành cấu hình được chọn.

Thành phần	Hình dung đời thường	Trong dự án
Model	Người có kiến thức	Gemma 4 12B
Engine	Bàn làm việc và dụng cụ	Ollama và backend của nó
Phần cứng	Không gian và năng lực làm việc	CPU, RAM, T4, VRAM
Bộ tối ưu	Người tổ chức công việc	Mã Python do nhóm phát triển
Benchmark	Bài kiểm tra có quy tắc	Cùng đề, cùng điều kiện, có số đo

Câu hỏi nghiên cứu cụ thể: với cùng model, context và yêu cầu sinh, cấu hình nào giảm thời gian trả lời trong ngân sách VRAM, đồng thời giữ điểm tác vụ trong ngưỡng cho phép?

Sản phẩm cuối là ba notebook có thể chạy, mã nguồn dùng chung, bộ dự đoán độ trễ được train từ số đo, cấu hình triển khai, dữ liệu thô và bản đánh giá. Khi dùng demo, bạn thay biến `PROMPT` rồi chạy ô hỏi đáp.

“Chạy trong Colab” nghĩa là GPU ở máy chủ Google. Máy cá nhân của bạn mở trình duyệt và gửi lệnh điều khiển; T4 làm phần tính toán. Sau này có thể chuyển mã điều phối sang máy cá nhân, nhưng phải đo lại để chọn cấu hình mới.

Bạn cần hiểu được sau trang này: đối tượng được tối ưu là toàn bộ quá trình suy luận của một hệ thống đã có model, không phải một điểm số AI chung chung.

02. Đọc notebook gốc như đọc một quy trình

Các chỉ số ô dưới đây đếm từ 0 theo tệp JSON của notebook gốc. Tệp có 27 ô, gồm ô văn bản, ô mã và một số ô trống.

Ô	Mã gốc làm gì?	Điều cần hiểu hoặc sửa
1-2	Cài Ollama, zstd, công cụ nhận diện phần cứng	Chưa phải tải trọng số Gemma
3	Cài thư viện openai	Không được dùng trong các lời gọi phía sau
5	Chạy nvidia-smi	Output lưu sẵn cho thấy T4, 15360 MiB
8	Khởi động ollama serve	Cần kiểm tra readiness và quản lý tiến trình
10	Gọi localhost	Chỉ chứng minh server trả lời
12	Cài Python SDK ollama	SDK là công cụ gọi API, không phải model
13	Ghi lệnh ollama pull trong Markdown	Run all không thực thi lệnh này
17	Hỏi vì sao bầu trời xanh	Demo văn bản, chưa có phép so sánh
21	Đọc ảnh rồi gửi images	Tệp ảnh phải thật sự tồn tại
22-23	Đọc audio rồi đưa base64 vào content	API nhận văn bản base64 trong đoạn mã này

Output trong notebook là **kết quả được lưu từ một lần chạy trước**, không phải bằng chứng rằng môi trường hiện tại đã có model. Ví dụ `ollama list` được lưu với danh sách rỗng, trong khi ô sau có câu trả lời; không thể tái dựng chắc chắn thứ tự thao tác ngoài notebook từ các output đó.

Bản mới khắc phục việc thiếu bước tải, dùng endpoint riêng `127.0.0.1:11435`, ghi log ra tệp và chờ server thật sự sẵn sàng. Không dùng PIPE không được đọc cho một server chạy dài: log đầy bộ đệm có thể khiến tiến trình chờ.

Không cần công khai `0.0.0.0` để Python trong cùng Colab gọi Ollama. Cũng không cần API key OpenAI cho lời gọi model local này.

Bài tập nhỏ: chỉ ra dòng nào tải engine, dòng nào tải model và dòng nào thực sự sinh câu trả lời. Nếu chưa phân biệt được ba việc này, quay lại bảng trên.

03. Token, trọng số và ý nghĩa của 12B

Máy không đọc câu hỏi theo đúng cách con người tách từ. Một tokenizer chuyển văn bản thành các mã số gọi là token. Một token có thể tương ứng một từ, một phần từ, dấu câu hoặc phần biểu diễn khác. Không được mặc định một token bằng một từ tiếng Việt.

Ví dụ minh họa “Tôi học Python” có thể được chia thành nhiều mảnh. Cách chia chính xác phụ thuộc tokenizer; cẩm nang không gán mã token tưởng tượng cho model. `eval_count` do engine trả về mới là số token sinh dùng trong bài đo.

Trọng số là rất nhiều con số model đã học từ trước. Chúng tham gia các phép nhân, cộng và phép biến đổi để tính token tiếp theo. Tên 12B nói đến quy mô khoảng 12 tỷ tham số; metadata của artifact mới là nơi xác nhận chi tiết thực tế.

Ước lượng để hiểu dung lượng, không phải VRAM đo thật:

- Khoảng 12 tỷ số, mỗi số 2 byte: khoảng 24 tỷ byte, xấp xỉ 22,35 GiB.
- Nếu trung bình 4 bit cho mỗi số: phần số học lý tưởng khoảng 6 tỷ byte, xấp xỉ 5,59 GiB.
- Thực tế còn scale lượng tử hóa, một số tensor chính xác cao, metadata, bộ đệm, cache và các thành phần bổ sung.

Vì thế không thể kết luận “12B 4-bit chỉ cần đúng 6 GB VRAM”. Cũng không thể nạp toàn bộ phần trọng số FP16 ước lượng ở trên vào riêng vùng VRAM 15360 MiB trong output T4 của bạn.

Trang model Ollama hiện có artifact `gemma4:12b` cho backend llama.cpp với trọng số Q4_K_M và một draft đi kèm. Điều đó xác nhận tag không đồng nghĩa với “FP16 nguyên gốc”. Notebook lưu `/api/show` cùng digest của bản thực sự tải. [S1]

Tình huống: tải model 8 GB không có nghĩa lúc chạy chỉ tốn 8 GB. Giống một quyển sách chiếm một ngăn tủ nhưng khi làm bài bạn còn cần bàn để mở sách, giấy nháp và tài liệu đang dùng.

04. Một câu trả lời được sinh như thế nào?

Bạn gửi “Viết ví dụ vòng lặp for”. Pipeline có hai giai đoạn cần phân biệt.

Prefill: engine xử lý phần đầu vào: system prompt, câu hỏi, lịch sử hội thoại và nội dung khác nếu có. Model tạo các biểu diễn cần thiết và thông tin cache. Prompt càng nhiều token, phần việc này thường càng lớn.

Decode: model sinh lần lượt các token mới. Tại bước t, model tính phân phối xác suất của token tiếp theo dựa trên prompt và những token đã sinh. Bộ chọn token quyết định token nào được nối vào câu trả lời.

Viết ngắn bằng công thức: **p(token tiếp theo | prompt, các token trước đó)**. Sau khi chọn một token, điều kiện ở bước sau đã thay đổi. Đây là lý do việc sinh thông thường có tính tuần tự.

Trong benchmark, `temperature=0` đưa quá trình chọn về chế độ tham lam thông thường: chọn token có điểm cao nhất. `seed=42` giúp kiểm soát nguồn ngẫu nhiên nhưng không bảo đảm bit-identical giữa mọi GPU, backend và phép tính lượng tử hóa.

Ví dụ minh họa: câu trả lời cần 150 token. Nếu prefill mất 1 giây và decode trung bình 25 token/giây thì tổng phần việc gần 1 + 150/25 = 7 giây, chưa tính nạp model, hàng đợi và truyền kết quả. Nếu tăng decode lên 30 token/giây, tổng gần 6 giây. Nếu prompt dài khiến prefill mất 10 giây, cùng thay đổi decode chỉ đưa tổng từ 16 xuống 15 giây.

Do đó câu hỏi “model nhanh bao nhiêu?” phải có ngữ cảnh: prompt ngắn hay dài, output ngắn hay dài, đã nạp model hay chưa?

Streaming chỉ đưa các phần trả lời tới người dùng sớm hơn. Nó cải thiện cảm giác chờ và giúp đo thời gian đầu ra đầu tiên; bản thân việc bật streaming không chứng minh engine tính toán nhanh hơn.

Tự kiểm tra: câu hỏi có 2.000 token đầu vào và câu trả lời 5 token thường nhạy với prefill; câu hỏi 30 token nhưng cần viết 500 token thường nhạy với decode. Đây là suy luận về khối lượng công việc, vẫn cần đo.

05. T4 bị giới hạn ở đâu?

GPU làm nhiều phép toán song song. Nhưng model còn phải đọc trọng số, đọc và ghi cache, đồng bộ tác vụ và đôi khi chuyển dữ liệu giữa RAM với VRAM. Nhiều năng lực tính toán không có nghĩa mọi bài đều nhanh tương ứng.

Compute-bound là trường hợp thời gian bị giới hạn chủ yếu bởi số phép toán cần thực hiện. **Memory-bound** là trường hợp việc chuyển dữ liệu qua các tầng bộ nhớ giữ vai trò lớn. Batch nhỏ ở giai đoạn decode thường đáng kiểm tra về băng thông bộ nhớ; không nên kết luận chỉ từ tên GPU. [S8]

Hình dung một đầu bếp rất nhanh nhưng phải đi sang kho lấy nguyên liệu cho từng món. Nâng tốc độ thao tác không giải quyết hết thời gian đi lấy hàng. Các kỹ thuật như nén trọng số, bố trí tính toán attention và sinh nhiều token được kiểm tra cùng lượt đều có liên quan tới cách sử dụng dữ liệu.

Nguồn tài nguyên	Dùng cho việc gì?	Cách kiểm tra
Ổ đĩa	Lưu model và kết quả	Kích thước tệp, dung lượng trống
RAM	Tiến trình, dữ liệu, phần model offload	Bộ đo host và RSS của cây tiến trình
VRAM	Trọng số trên GPU, KV cache, bộ đệm	nvidia-smi và bộ lấy mẫu
CPU	Điều phối, tokenization, phần chạy CPU	Hồ sơ môi trường và log

Ngân sách mặc định của dự án là **13824 MiB = 13,5 GiB**, chừa khoảng trống so với 15360 MiB trong output gốc. Đây là ràng buộc tự đặt cho thí nghiệm, không phải mức tiêu thụ dự kiến hay kết quả đã đạt.

Model vừa đủ lúc nạp vẫn có thể hết bộ nhớ khi context tăng. Nếu engine chuyển một phần tính toán sang CPU, chương trình có thể chạy được nhưng thời gian đổi khác đáng kể. Lưu `/api/ps` và log để hiểu nơi model được nạp; dùng số đo thiết bị để đánh giá VRAM.

Điểm quan trọng: `torch.cuda.max_memory_allocated()` trong notebook không theo dõi toàn bộ bộ nhớ của tiến trình Ollama độc lập. Bộ đo này vì vậy lấy số liệu từ ngoài tiến trình model.

06. “Chưa tối ưu” phải được định nghĩa công bằng

Nếu một chiếc xe xuất xưởng đã có hộp số tự động, bạn không thể gọi nó là xe “chưa có hộp số”. Tương tự, model Ollama có thể đã lượng tử hóa, có KV cache, tự chọn kernel và có draft. Phần tối ưu của nhóm phải đo lợi ích tăng thêm trên nền đó.

Trong gói này có hai lớp đối chiếu:

1. **Demo gốc:** giữ hình thức hỏi đáp ban đầu để kiểm tra chức năng. Nó không phải mốc công bố tốc độ vì các mặc định có thể khác.
2. **B0_original_controlled:** giữ artifact gốc và các tối ưu tự động của engine, nhưng cố định yêu cầu sinh để so các cấu hình công bằng.

Điều kiện	Giá trị trong bài đo chính
Model	gemma4:12b, cùng digest
Context	4096 ở mọi ứng viên
Sampling	temperature 0, seed 42, cùng các tham số khác
Thinking	think=False ở mọi ứng viên
Số yêu cầu đồng thời	1
Output cap	128 hoặc 256, theo từng prompt
Flash Attention ở B0	Tự động theo engine
Draft ở B0	Không ghi đè mặc định của model

Không tắt cache hoặc cố ý đưa model lên CPU rồi lấy cấu hình chạy GPU bình thường để tuyên bố một mức tăng tốc ấn tượng. Có thể nghiên cứu các đối chứng đó riêng, nhưng tên và mục đích phải rõ.

`think=False` là một điều kiện kiểm soát; không được so B0 bật thinking với bản tối ưu tắt thinking rồi quy toàn bộ lợi ích cho cache. Việc tắt thinking cũng không bảo đảm phù hợp mọi bài suy luận; đồ án này chọn một phạm vi text cụ thể. [S5]

Câu có thể dùng trong báo cáo: “Baseline là bản Gemma 4 12B do Ollama phân phối, với các mặc định engine được giữ trong phạm vi nêu rõ; các điều kiện đầu vào và sinh được kiểm soát giống nhau.”

07. Bản đồ ba Colab và các tệp đi qua chúng

Giai đoạn	Bạn làm gì?	Sản phẩm chuyển sang bước sau
01 - Baseline	Nhận diện T4, tải model, khóa phiên bản, đo ban đầu	experiment_lock.json, model_show.json, raw.jsonl
02 - Train & Optimize	Thử ứng viên, thu số đo, train predictor, chọn cấu hình	latency_predictor.joblib, selection.json, candidate_audit.json
03 - Evaluate & Demo	Khóa policy, chạy test cùng runtime, so sánh và sử dụng	comparison.csv, paired_speedup.json, biểu đồ, demo

Mỗi notebook tự mang theo cùng mã dùng chung. Ô đầu giải nén mã vào `/content/gemma4_t4_code`; bản dễ đọc nằm trong thư mục `src/` của ZIP. Việc nhúng mã giúp bạn không phải upload nhiều tệp phụ trước khi học.

Drive giữ số đo, log và bộ chọn đã train. Thư mục `/content/ollama_models` giữ model lớn trong runtime. Khi runtime bị xóa, model có thể phải tải lại; notebook so digest để phát hiện bản tải sau đã khác.

Vì sao phải có khóa phiên bản? Tên `gemma4:12b` là một nhãn. Chủ phân phối có thể cập nhật nội dung dưới nhãn đó. Digest giống dấu vân tay của nội dung. Nếu engine hoặc model đổi, kết quả trước và sau không còn chỉ khác ở tham số tối ưu.

`EXPERIMENT_ID` là tên một đợt thí nghiệm. Dùng cùng ID ở 01, 02, 03. Muốn đổi bộ dữ liệu, context hay ngân sách, hãy tạo một đợt mới và đo lại từ đầu. Không sửa khóa chỉ để cho chương trình chạy qua.

Các phép đo ghi từng dòng ngay sau một yêu cầu, nên phần đã hoàn thành có thể còn trên Drive sau khi Colab ngắt. Mã hiện tại **không tự nối tiếp nửa ma trận bị gián đoạn**; lần chạy lại tạo thư mục mới. Chỉ sử dụng bộ đủ các cặp đo cho lựa chọn cuối cùng.

Colab thay đổi tài nguyên khả dụng giữa các phiên, vì vậy notebook 03 luôn chạy lại các đối chứng trong chính runtime đang đánh giá. [S10]

08. Quantization: nén con số nào?

Quantization là biểu diễn một nhóm số bằng độ chính xác thấp hơn cùng thông tin để xấp xỉ lại chúng. Hình dung làm tròn một bảng giá: ghi 12,37 bằng một biểu diễn ngắn hơn tiết kiệm chỗ, nhưng không giữ nguyên toàn bộ chi tiết.

Trong LLM có ít nhất hai đối tượng khác nhau: **trọng số** và **KV cache**. Model có trọng số Q4 không buộc cache của nó cũng phải Q4. Đây là hai lựa chọn độc lập cần ghi riêng.

Đối tượng	Có sẵn hay được thay đổi trong gói này?
Trọng số Gemma	Giữ nguyên artifact đã tải
KV cache	Thử f16 và q8_0 theo khả năng engine
Draft	Dùng draft đi kèm khi backend hỗ trợ
Predictor độ trễ	Train từ dữ liệu benchmark của bạn

Gói này không tạo checkpoint Gemma Q4 mới từ FP16. Vì vậy nếu VRAM giảm giữa B0 và C2, không được viết “nhóm lượng tử hóa toàn bộ model từ 16 xuống 4 bit”. Thay đổi đang thử là kiểu biểu diễn cache và cấu hình chạy.

Ví dụ số học giả định: trước thay đổi, trọng số và phần cố định dùng 8 GiB, cache dùng 2 GiB, phần khác 1 GiB. Tổng là 11 GiB. Nếu cache giảm còn khoảng 1 GiB, tổng gần 10 GiB: tiết kiệm khoảng 9,1% tổng bộ nhớ, không phải 50%. Con số 50% chỉ nói về phần cache trong giả định này.

Nén ít bit hơn cũng thêm chi phí giải mã và có thể thay đổi kết quả số học. Tốc độ cuối cùng phụ thuộc kernel, phần cứng và tác vụ. “Nhỏ hơn” và “nhanh hơn” là hai câu hỏi phải đo riêng.

Nếu muốn bổ sung lượng tử hóa trọng số thành nghiên cứu sau này, cần một checkpoint nguồn xác định, công cụ hỗ trợ đúng kiến trúc và các bản nén đối chứng. Không nén tiếp một bản Q4 rồi xem nó tương đương nén trực tiếp từ bản độ chính xác cao.

09. KV cache: cuốn sổ ghi nhớ tạm thời

Trong attention, mỗi token có các biểu diễn được gọi là key và value. KV cache giữ lại các biểu diễn cần dùng từ những bước trước, giúp model tránh tính lại toàn bộ phần tương ứng khi sinh token mới.

So sánh với làm một bài toán nhiều bước: bạn giữ kết quả trung gian trên giấy nháp để dùng tiếp. Giấy nháp đó không thay đổi kiến thức lâu dài của bạn. Tương tự, KV cache không phải "huấn luyện model từ cuộc trò chuyện".

Với kiến trúc attention đơn giản, một ước lượng là:

Bộ nhớ KV ≈ 2 × số lớp × token lưu × số KV head × head dimension × byte mỗi số × số sequence.

Số 2 xuất hiện vì có cả key và value. Đây là công thức giảng giải; Gemma có các chi tiết kiến trúc và cửa sổ attention riêng, nên không dùng công thức đơn giản để công bố số VRAM chính xác của Gemma 4 12B.

Trong Ollama, biến môi trường `OLLAMA_KV_CACHE_TYPE` chọn loại cache. Giá trị ứng viên trong mã là `f16` hoặc `q8_0`. Tài liệu hiện yêu cầu Flash Attention cho cache lượng tử hóa và lưu ý chất lượng phụ thuộc tác vụ. [S2]

Tình huống 1: hỏi một câu ngắn. Cache nhỏ so với trọng số, nên giảm cache có thể chỉ tiết kiệm ít tổng bộ nhớ.

Tình huống 2: đưa một đoạn tài liệu dài vào câu hỏi. Cache và chi phí attention đáng quan tâm hơn; nhưng phải giữ nguyên tài liệu giữa hai cấu hình.

Tình huống 3: hội thoại 20 lượt. Lịch sử cũng chiếm context. Demo của gói dùng câu hỏi độc lập để tránh việc bạn vô tình so hai lịch sử khác nhau.

Không giảm `num_ctx` từ 8192 xuống 2048, cắt mất tài liệu rồi báo hệ thống nhanh hơn mà giữ nguyên chức năng. Nếu người dùng thật sự chấp nhận context ngắn hơn, đó là một đánh đổi sản phẩm cần ghi rõ và đánh giá lại.

10. Flash Attention và kích thước batch

Flash Attention tổ chức phép tính attention theo cách giảm việc chuyển dữ liệu giữa các tầng bộ nhớ, thay vì lưu tất cả các trung gian lớn như một cách triển khai thông thường. Ý tưởng trọng tâm là cách sắp xếp tính toán và truy cập dữ liệu. Không thể suy từ tên kỹ thuật ra một hệ số tăng tốc cố định cho T4. [S7]

Trong gói này, `OLLAMA_FLASH_ATTENTION=1` là yêu cầu gửi cho engine trước khi khởi động. Nếu backend không hỗ trợ hoặc đã tự bật từ B0, cấu hình C1 có thể không tạo khác biệt. **Đặt một biến môi trường không chứng minh một kernel đã chạy.**

Notebook lưu log cho từng khối cấu hình. Khi viết báo cáo, tách "giá trị yêu cầu" khỏi "hành vi xác nhận trong log". Nếu log không đủ rõ, chỉ báo kết quả của cấu hình yêu cầu, không quy nguyên nhân chắc chắn cho Flash Attention.

`num_batch` là kích thước nhóm token xử lý prompt mà engine sử dụng theo khả năng hỗ trợ. Nó không phải "số người chat cùng lúc". `OLLAMA_NUM_PARALLEL` mới liên quan đến số yêu cầu đồng thời; bài đo này giữ bằng 1.

Ví dụ: khi xử lý một tài liệu, một batch lớn có thể đưa nhiều việc vào GPU hơn và cải thiện prefill, nhưng cần thêm bộ đệm. Batch nhỏ có thể giảm bộ nhớ tạm hoặc giảm hiệu quả GPU. Với output dài, decode có thể vẫn chiếm phần lớn thời gian nên thay batch không tạo thay đổi tương ứng ở tổng thời gian.

Ứng viên C3 giảm batch từ 512 xuống 256 trên nền Flash Attention + cache Q8. C2 và C3 là cặp phù hợp để khảo sát thay đổi đó. Không so C3 với một cấu hình đồng thời đổi context và sampling rồi cho rằng tất cả khác biệt do batch.

Không cần cài gói Python `flash-attn` để thực hiện thí nghiệm này. Ollama dùng backend riêng; tên chung "Flash Attention" không khiến thư viện Python và kernel của engine trở thành cùng một đường thực thi.

11. Speculative decoding: viết nháp rồi kiểm tra

Khi sinh thông thường, model lớn tính một bước để lấy token tiếp theo. Speculative decoding dùng một thành phần nhỏ hơn đề xuất nhiều token, rồi model lớn kiểm tra đoạn đề xuất trong một lượt thích hợp.

Ví dụ token tượng trưng: draft đề xuất [A, B, C, D]. Model chính đồng ý A, B nhưng tại vị trí tiếp theo chọn X. Hệ thống giữ phần hợp lệ, xuất token sửa và bỏ phần còn lại không còn đúng điều kiện. Token không phải từ nguyên vẹn; ví dụ chỉ giúp nhìn ra logic.

Với greedy decoding, có thể hình dung kiểm tra theo lựa chọn argmax của target. Với sampling, không đơn giản "cùng chữ thì nhận": thuật toán chính xác cần quy tắc xác suất chấp nhận và phân phối hiệu chỉnh để bảo toàn phân phối mục tiêu. [S6]

Phép tính giả định: chạy target tuần tự tốn 40 ms/token. Một lượt draft tốn 15 ms, target kiểm tra tốn 50 ms, điều phối 5 ms. Nếu lượt đó xuất 3 token, chi phí khoảng 70/3 = 23,3 ms/token. Nếu chỉ xuất 1 token, chi phí 70 ms/token và chậm hơn ban đầu.

`draft_num_predict=6` là giới hạn token nháp được yêu cầu, không có nghĩa luôn xuất 6 token mỗi lượt. Draft phải phù hợp model và backend. Không thể ghép tùy tiện một model 0,5B bất kỳ với Gemma rồi mặc định chạy được.

Artifact Gemma hiện được Ollama phân phối có draft đi kèm; baseline mới có thể đã dùng nó. Mã thử mặc định, tắt draft bằng 0 và tăng giới hạn lên 6. Tài liệu Modelfile xác nhận tham số và ý nghĩa tắt draft. [S1, S4]

API benchmark hiện sử dụng không cung cấp trực tiếp số drafted/accepted theo từng vòng. Gói không bịa acceptance rate từ tốc độ hoặc số token xuất. Muốn nghiên cứu tỷ lệ đó, cần đo trong engine hoặc log có bộ đếm rõ nghĩa.

12. Train, fine-tune và autotune khác nhau thế nào?

Khái niệm	Thay đổi cái gì?	Ví dụ
Inference	Không cập nhật trọng số	Gemma trả lời câu hỏi
Fine-tuning	Cập nhật một phần/toàn bộ trọng số model	Học cách trả lời theo dữ liệu chuyên ngành
Autotuning	Chọn tham số vận hành bằng thử nghiệm	Chọn cache và batch
Train predictor	Học quan hệ từ số đo	Dự đoán cấu hình nào trả lời nhanh hơn

Colab 2 của gói này có train predictor. Đầu vào là đặc trưng yêu cầu và cấu hình. Nhãn là độ trễ đo thật. Tệp sau train là `latency_predictor.joblib`. Toàn bộ trọng số Gemma được giữ nguyên.

Tại sao chọn cách này? Bạn đang dùng Ollama làm hệ thống suy luận, và mục tiêu trước mắt là cải thiện cách vận hành trên T4. Huấn luyện thêm kiến thức cho Gemma không tự động giảm số phép tính khi nó sinh mỗi token.

LoRA học các ma trận bổ sung kích thước nhỏ thay vì cập nhật mọi trọng số. QLoRA kết hợp cách huấn luyện hiệu quả với model nền được lượng tử hóa. Chúng có thể hữu ích để sửa hành vi hoặc thích nghi miền dữ liệu, nhưng không mặc nhiên là giải pháp tăng tốc inference.

Nếu yêu cầu của giảng viên là “phải fine-tune chính LLM”, gói hiện tại chưa đáp ứng hạng mục đó. Cần bổ sung một nhánh riêng: checkpoint Gemma được hỗ trợ, dữ liệu có quyền dùng, cấu hình huấn luyện vừa T4, adapter/checkpoint, chuyển đổi và kiểm chứng lại trên engine. Khả năng train model đa phương thức 12B phải được thử về bộ nhớ; không lấy việc model Q4 chạy inference được làm bằng chứng train chắc chắn vừa.

Cách giới thiệu đúng: “Nhóm giữ nguyên Gemma, huấn luyện mô hình dự đoán chi phí suy luận và dùng nó hỗ trợ chọn cấu hình dưới ràng buộc bộ nhớ, chất lượng.” Không viết “đã train lại Gemma 12B” khi chỉ tạo predictor.

13. Bộ dự đoán học từ dữ liệu gì?

Mỗi dòng benchmark giống một quan sát: “Câu hỏi thuộc loại A, dài B byte, yêu cầu tối đa C token, chạy cấu hình D, mất E giây”. Sau khi có nhiều quan sát, model nhỏ học cách ước lượng E từ A, B, C, D.

Ví dụ **giả định**, không phải log T4:

Prompt	Byte đầu vào	Cap	Cấu hình	Thời gian
Câu hỏi ngắn	260	128	B0	4,8 s
Câu hỏi ngắn	260	128	C2	4,5 s
Viết tài liệu	1800	256	B0	11,2 s
Viết tài liệu	1800	256	C2	9,6 s

Byte là đặc trưng đo dễ trước khi sinh; nó không thay cho token trong báo cáo tốc độ. `task` do bộ bài thử hoặc ứng dụng khai báo. Tệp model dùng `DictVectorizer` chuyển đặc trưng dạng tên/giá trị thành mảng số.

Random Forest kết hợp nhiều cây quyết định. Mỗi cây đặt những điều kiện trên các đặc trưng để ước lượng đầu ra. Ta lấy ước lượng kết hợp của chúng. Gói thử hai giới hạn độ sâu cây 3 và 6, mỗi mô hình gồm 128 cây; mục tiêu học là log(thời gian), sau đó dùng hàm mũ để đưa dự đoán về giây. [S9]

Không đưa `eval_count` thực, `wall_s` hay VRAM sau khi sinh vào đặc trưng đầu vào. Làm như vậy sẽ dùng thông tin mà lúc cần quyết định chưa có - một dạng rò rỉ dữ liệu.

Các lần đo lặp của cùng prompt/cấu hình được gom bằng trung vị trước khi train. MAE trên validation cho biết sai số dự đoán tính theo giây. Sai số thấp chưa chứng minh chọn cấu hình tốt: vẫn phải so thời gian của cấu hình được chọn với best-fixed.

Predictor nhỏ chạy bằng CPU; không cần dùng T4 để train nó. T4 được dùng để tạo dữ liệu chi phí chạy Gemma. Không có dữ liệu đo thật thì predictor của dự án chưa được train.

14. Quy tắc chọn cấu hình và giới hạn của nó

Trước khi hỏi cấu hình nào nhanh, hệ thống loại các ứng viên không hợp lệ. Một ứng viên cần chạy đủ bộ validation, không lỗi runtime, có số đo VRAM, không vượt ngân sách và không làm giảm điểm JSON quá ngưỡng.

Ngưỡng giảm điểm mặc định là **0,05 điểm trên thang 0-1**, tức 5 điểm phần trăm, không phải giảm tương đối 5%. Nếu baseline đạt 0,90 thì ngưỡng là 0,85. Bộ mẫu ít có bước điểm lớn, nên ngưỡng này có thể tương đương không được sai thêm câu nào.

Những bài tốc độ phải sinh đủ token cap, không có thinking ngoài điều kiện đặt ra. Nếu sinh thiếu hoặc có lỗi, cấu hình không tự được lợi vì xử lý ít việc hơn.

Trong các ứng viên còn lại, có hai cách chọn:

- **Best-fixed:** chọn cấu hình có thời gian trung bình validation thấp nhất.
- **Predictor:** dự đoán chi phí trên nhóm yêu cầu validation rồi chọn cấu hình có chi phí dự đoán thấp nhất.

Nếu lựa chọn predictor chậm hơn best-fixed quá 3% trên validation, dùng best-fixed. Khi baseline vẫn hợp lệ, nhánh predictor còn phải có lợi ít nhất 5% so với baseline trên validation. Đây là quy tắc thiết kế, không phải bằng chứng thống kê chắc chắn.

Giới hạn chủ ý: cấu hình được chọn một lần cho một phiên/nhóm công việc. Gói không thay cache giữa mỗi token và không triển khai bộ điều khiển vòng speculative bên trong Ollama. Đây là hệ thống tối ưu cấu hình có predictor, chưa phải một thuật toán decoding mới.

Vì best-fixed đã được chọn từ toàn bộ validation, predictor không có lý do mặc nhiên thắng best-fixed trên chính tập đó. Đóng góp của predictor chỉ thuyết phục nếu nghiên cứu tiếp cho thấy chọn tốt trên workload mới hoặc giảm số phép dò cấu hình cần đo. Gói hiện tại vẫn thu ma trận ứng viên đầy đủ; chưa chứng minh giảm chi phí tìm kiếm.

Tình huống: nếu C2 luôn tốt nhất cho cả nhóm câu hỏi, chọn cố định C2 là một kết quả hợp lý. Không cần ép dùng model dự đoán chỉ để hệ thống trông phức tạp.

15. Thiết kế dữ liệu: train, validation, test

Hãy hình dung luyện thi: train là bài để học; validation là bài để điều chỉnh cách học; test là đề cuối cùng chưa dùng để điều chỉnh. Nếu xem đáp án test rồi đổi phương pháp, điểm test không còn là kiểm tra độc lập.

Chế độ	Train	Validation	Test	Mục đích
QUICK=True	12	8	8	Kiểm tra quy trình
QUICK=False	24	12	16	Đợt thử kỹ hơn, vẫn là pilot

Bộ mặc định có bốn nhóm: bài tính giá trị cần JSON, truy tìm dữ kiện cần JSON, giải thích dài và đầu vào dài yêu cầu viết nhiều đoạn. Dữ liệu do mã tạo cố định theo seed để chạy lại được.

Đây là benchmark kỹ thuật tổng hợp, không phải bộ chuẩn đánh giá hiểu biết tiếng Việt hay khả năng lập trình tổng quát. Train và test có nội dung khác nhau nhưng vẫn dùng các họ mẫu chung. Kết quả vì vậy chủ yếu chứng minh trên phân phối bài thử hẹp này.

Để làm báo cáo mạnh hơn, bổ sung 50-100 câu hỏi thực tế đã phân nhóm, với đáp án hoặc rubric rõ ràng. Nếu dùng nhiều câu từ cùng một tài liệu, chia tập theo tài liệu trước; không để các đoạn gần giống nhau của một tài liệu nằm ở cả train lẫn test.

Nhóm tác vụ gợi ý: hỏi đáp theo thông báo câu lạc bộ; giải thích mã Python; tóm tắt một đoạn văn tự viết; trích xuất phòng, giờ, hạn nộp từ văn bản. Giữ tập đánh giá cuối không dùng trong chọn cấu hình.

Khi bổ sung dữ liệu, xác định trước tiêu chí chấm. Chấm đúng JSON là kiểm tra trích xuất/tính toán có định dạng; không đại diện cho độ dễ hiểu của một bài giải thích. Các bài giải thích trong bộ mặc định phục vụ đo tải sinh, không được tự nhận điểm chất lượng chỉ vì trả lời dài.

Nguyên tắc: muốn tuyên bố điều gì, cần bộ dữ liệu đo đúng điều đó. Muốn tuyên bố "tốt cho học sinh mới học Python", phải có người chấm hoặc kiểm tra đầu ra phù hợp mục tiêu đó.

16. Benchmark đo các đại lượng nào?

Đại lượng	Cách hiểu	Trong mã
Thời gian nạp	Đưa model vào trạng thái chạy	load_s
Thời gian toàn yêu cầu	Từ trước gửi HTTP đến hết stream	wall_s
Đầu ra nội dung đầu tiên	Khi nhận mảnh content không rỗng đầu	ttft_content_s
Đầu ra sinh đầu tiên	Content hoặc thinking đầu tiên	ttft_generated_s
Tốc độ engine sinh	eval_count chia eval_s	decode_tps_engine
Throughput yêu cầu	eval_count chia wall_s	end_to_end_tps
Đỉnh VRAM lấy mẫu	Giá trị cao nhất quan sát trên GPU	peak_gpu_mib
Đỉnh RSS lấy mẫu	Tổng RSS cây tiến trình server	peak_server_rss_mib

API Ollama trả thời gian engine theo nanosecond. Đổi sang giây bằng chia 1.000.000.000. Ví dụ 128 token trong 4.000.000.000 ns cho tốc độ 32 token/giây. Không chia nhầm trực tiếp cho số nanosecond. [S3]

TTFT đo ở HTTP là **xấp xỉ theo mảnh stream đầu tiên**, không phải bộ đếm phần cứng bên trong kernel. Một chunk có thể chứa nhiều token. Gói không lấy số chunk làm số token và không công bố inter-token latency từ khoảng cách chunk.

`eval_count` có thể bao gồm lượng sinh ngoài content nếu bật thinking. Bài đo cố định think=False và vẫn ghi số ký tự thinking nhận được để phát hiện điều kiện không như yêu cầu. Không xem “chưa có content” là TTFT bằng 0.

VRAM được lấy mẫu khoảng mỗi 0,25 giây qua nvidia-smi. Đây là đỉnh quan sát, có thể bỏ lỡ đỉnh rất ngắn; còn tính cả tài nguyên tiến trình khác trên GPU. Tổng RSS cũng có thể đếm trùng trang bộ nhớ chia sẻ. Các tên số đo thể hiện đúng giới hạn đó.

Prefill rate chỉ được tính theo token chưa cache khi API có trường cached count. Nếu thiếu trường cần thiết, giá trị để trống. Không điền 0 vào phép đo thiếu rồi gọi đó là kết quả tốt.

17. Quy trình đo để tránh tự đánh lừa mình

Bước 1 - Cố định bài toán. Cùng model/digest, template, prompt, context, cap, sampling và thinking. Ghi môi trường, phiên bản engine và thư viện.

Bước 2 - Phân biệt lần nạp đầu và model đã nạp. Bộ đo chính dùng model đã warm-up. Thời gian khởi động server và warm-up được lưu riêng trong setup. Demo cold/warm ở notebook 01 giúp thấy sự khác biệt.

Bước 3 - Tránh tái sử dụng toàn bộ prompt vô tình. Mỗi prompt/lần lặp có mã lượt đo khác, nhưng mã đó giống nhau giữa các cấu hình được so. Vẫn có thể tái sử dụng prefix template chung; số token cache được lưu nếu API cung cấp. Không gọi đây là “cache hoàn toàn rỗng”.

Bước 4 - Đảo thứ tự. Mã xáo trộn các khối cấu hình theo seed ở mỗi lần lặp để giảm ảnh hưởng việc một cấu hình luôn chạy đầu, một cấu hình luôn chạy cuối. Mỗi khối dùng một cấu hình và có warm-up riêng.

Bước 5 - Đo lặp và giữ lỗi. Không xóa OOM, output rỗng, thiếu done=true hoặc cấu hình bị từ chối. Mất một số cặp có thể làm so sánh lệch.

Bước 6 - Tách bài tốc độ khỏi bài chất lượng. `num_predict` chỉ là giới hạn tối đa; model có thể dừng trước. Bài tốc độ chỉ hợp lệ cho so độ dài cố định khi số token thực bằng cap. Bài chất lượng cho phép dừng tự nhiên, nhưng output bị cắt do length sẽ không được tính là đúng.

Bước 7 - Đo lại trên test trong cùng runtime. Dùng lựa chọn đã khóa từ notebook 02. Không lấy kết quả notebook 01 trên T4 phiên trước làm đối chứng duy nhất.

Bước 8 - Phân tích theo nhóm. Báo prompt ngắn/dài, output ngắn/dài. Trung vị chung trộn tác vụ có thể che việc một nhóm nhanh hơn trong khi nhóm khác chậm đi.

Các bước trên giúp phép đo có ý nghĩa; chúng không xóa hết nhiễu từ nhiệt độ, GPU cấp phát, tác vụ nền và cache hệ thống. Pilot nhỏ cần được mô tả đúng là pilot.

18. Đọc một bảng kết quả bằng số cụ thể

Toàn bộ số trong trang này là giả định để học cách tính, không phải kết quả T4. Giả sử ba cấu hình cùng sinh đủ 200 token trên một yêu cầu.

Cấu hình	TTFT	Tổng thời gian	Tốc độ decode	VRAM đỉnh
B0	1,5 s	10,0 s	25 token/s	12,0 GiB
C2	1,2 s	8,0 s	31 token/s	10,5 GiB
C5	1,1 s	11,0 s	22 token/s	11,0 GiB

C2 có speedup tổng thời gian **10/8 = 1,25 lần**. Nó giảm thời gian **(10-8)/10 = 20%**. Hai cách nói “1,25 lần” và “giảm 20%” tương ứng nhau, không phải tăng tốc 25% thì giảm thời gian 25%.

C2 giảm VRAM **(12-10,5)/12 = 12,5%**. C5 dùng ít bộ nhớ hơn B0 nhưng chậm hơn. Nếu mục tiêu là thời gian trong ngân sách đủ rộng, C2 phù hợp hơn trong ví dụ này.

Giả sử quality B0 là 18/20 và C2 là 17/20. Điểm giảm từ 0,90 xuống 0,85, bằng 5 điểm phần trăm. Có đạt ngưỡng hay không phụ thuộc ràng buộc đặt trước, không được chọn ngưỡng sau khi thấy bảng.

Trung vị là giá trị giữa sau khi sắp xếp các phép đo. Nó ít bị một lần rất chậm kéo lệch hơn trung bình. p95 là mức mà khoảng 95% quan sát không vượt qua; ít quan sát thì p95 rất thiếu ổn định.

Gói bootstrap theo **prompt**, sau khi lấy trung vị qua các lần lặp của từng prompt. Cách này tránh giả vờ các lần chạy lặp cùng một câu hỏi là những đề độc lập. Khoảng tin cậy rộng bao quanh 1,0 nghĩa là chưa có bằng chứng rõ về mức tăng tốc trong bộ mẫu đó.

Đừng chỉ đưa một biểu đồ đẹp. Kèm số cặp hợp lệ, số lỗi, độ dài output thực, điểm chất lượng và phạm vi dữ liệu. Nếu C2 chỉ thắng ở output dài, kết luận nên nói rõ nhóm đó.

19. Chạy notebook 01 từng bước

1. Giải nén ZIP trên máy. Mở Colab và upload `01_Baseline_Gemma4_T4.ipynb` từ thư mục notebooks.
2. Chọn Runtime → Change runtime type → T4 GPU. Kết nối runtime.
3. Chạy ô nạp mã dùng chung. Bạn không cần sửa chuỗi dữ liệu dài trong ô này.
4. Mount Google Drive. Giữ EXPERIMENT_ID mặc định cho lần học đầu; QUICK=True để thử quy trình.
5. Chạy ô cài engine. Cảnh báo systemd không hoạt động trong Colab không tự nó chứng minh model không chạy; server được khởi động bằng subprocess.
6. Kiểm tra tên GPU và VRAM thực. Ô xác minh tải model nếu thiếu rồi lưu phiên bản/digest.
7. Có thể bỏ qua demo gốc; nó chỉ để đối chiếu hình thức gọi, không tạo mốc tốc độ.
8. Chạy cold/warm demo, sau đó chạy baseline trên train/validation.
9. Mở summary và một vài dòng raw để kiểm tra output, số token và thời gian.
10. Chạy manager.stop(), rồi mở notebook 02.

Điều bạn cần quan sát: câu trả lời có nội dung không; `eval_count` có hợp lý không; load_s lớn ở lần đầu không; GPU có được sử dụng không; có dòng báo CPU offload hoặc cắt prompt không.

Một lần tải model thất bại có thể là mạng hoặc thiếu đĩa, chưa liên quan đến thuật toán decoding. Một lần server chạy nhưng model trả lỗi chưa chứng minh model đã sẵn sàng. Đọc loại lỗi trước khi đổi cấu hình.

Không chạy đồng thời notebook 01 và 02 để lấy benchmark cùng lúc. Hai server hoặc hai tác vụ dùng chung tài nguyên sẽ làm thay đổi điều kiện đo. Notebook chỉ quản lý tiến trình của chính nó; không dùng pkill để giết mọi tiến trình Ollama.

Bạn đã hoàn thành khi: có experiment_lock, model_show, cold_warm_demo và thư mục baseline chứa raw cùng manifest. Giá trị số là số máy bạn vừa đo, không phải những ví dụ giả định trong tài liệu.

20. Chạy notebook 02 và hiểu các đầu ra train

Mở notebook 02, chọn T4, giữ cùng EXPERIMENT_ID và QUICK. Các ô đầu đọc cấu hình và kiểm tra khóa đã có. Nếu model/engine hoặc dữ liệu khác, chương trình dừng thay vì trộn hai thí nghiệm.

Ô train cài thư viện cho predictor. Nếu môi trường Colab báo lỗi tương thích thư viện đã nạp trước, khởi động lại runtime và chạy lại các ô từ đầu; không bỏ qua một import lỗi rồi dùng đối tượng model cũ còn trong bộ nhớ.

Trước khi chạy ma trận, notebook in số lượt đo. QUICK dùng 20 prompt phát triển × 6 cấu hình × 1 lần = **120 lượt**, chưa kể warm-up. Bản đầy đủ dùng 36 × 6 × 2 = **432 lượt**. Thời gian phụ thuộc tốc độ thực, độ dài output và lỗi; đây không phải một thao tác vài giây.

Mỗi khối cấu hình khởi động lại engine và warm-up, sau đó đo các prompt. Đổi biến môi trường sau khi engine đã chạy không có hiệu lực hồi tố; đó là lý do phải khởi động lại khi thay cache/attention.

Tệp	Bạn đọc điều gì?
tuning_.../raw.jsonl	Mọi lượt chạy, kể cả lỗi
candidate_audit.json	Cấu hình nào bị loại và vì sao
training_log.json	Độ sâu cây, MAE validation
latency_predictor.joblib	Predictor đã train từ số đo
selection.json	Cấu hình chọn, best-fixed, ngưỡng
pip_freeze_train.txt	Các phiên bản thư viện thực tế

Nếu tất cả bị loại, chương trình chưa tìm được hệ thống đạt yêu cầu. Hãy xem nguyên nhân: thiếu token ở bài tốc độ, VRAM vượt ngưỡng, một option không hỗ trợ, điểm JSON giảm hay thiếu dữ liệu. Không tự nâng ngân sách chỉ để chọn được một dòng có màu xanh.

Gói có bộ kiểm tra byte bảo thủ để tránh gửi prompt quá dài. Nó không thay thế tokenizer chính xác. Bài đo còn đọc log cắt đầu vào; khi mở rộng context phải kiểm tra cả số token và hành vi engine.

Kết thúc: chạy manager.stop(). Bạn có thể giải thích tệp nào là predictor, tệp nào là trọng số Gemma, và vì sao chúng không phải cùng một đối tượng.

21. Chạy notebook 03 và sử dụng hệ thống

Notebook 03 đọc selection đã khóa, kiểm tra model/engine, rồi mới chạy tập test. Nó đo B0, best-fixed và lựa chọn của predictor. Nếu các lựa chọn giống nhau, chỉ cần đo mỗi cấu hình một lần trong ma trận.

QUICK đo 2 lần lặp, bản đầy đủ đo 3. Số lượng này đủ kiểm tra pipeline và quan sát ban đầu, chưa phải bằng chứng cho mọi điều kiện T4. Với báo cáo chính thức nên tăng số prompt thực trước khi chỉ tăng số lần lặp của vài prompt.

Sau phép đo, đọc `comparison.csv`, `paired_speedup.json`, `technical_gate.json` và hình speed_memory. Tên cột trong mã là tiếng Anh để dễ xử lý; ý nghĩa đã giải thích ở phần 16.

Phép kiểm tra kỹ thuật xét lỗi, VRAM và điểm chất lượng. Nếu cấu hình được chọn vi phạm, demo quay về B0 và báo rõ. Kết quả test của cấu hình tối ưu vẫn được giữ; không sửa selection bằng cách nhìn test rồi tuyên bố đó là đánh giá độc lập mới.

Dùng demo: thay `PROMPT`, giữ cap phù hợp rồi chạy ô. Câu trả lời xuất dần trên Colab. Một yêu cầu mới là câu độc lập; lịch sử hội thoại chưa tự được đưa lại vào prompt. Bạn có thể chạy lại ô với câu hỏi khác trong cùng phiên để giữ model đang nạp.

Lần khởi động demo sau benchmark có thể nạp model lại. Nếu lần đầu chậm, xem load_s. Các lần sau cần so cùng trạng thái model và độ dài câu hỏi trước khi kết luận cấu hình không hiệu quả.

Ô cuối tạo ZIP kết quả thật trên Drive và trong runtime để tải về. Nó chứa log của lần chạy máy bạn, khác với gói mã nguồn bàn giao ban đầu.

Vận hành lâu dài: dùng một cấu hình cho một phiên hoặc nhóm yêu cầu. Nếu chuyển cấu hình tốn 20 giây và tiết kiệm 0,5 giây mỗi câu, cần hơn 40 câu mới bù chi phí chuyển trong ví dụ giả định. Cấu hình nhanh khi model đã nạp chưa chắc tốt cho người chỉ hỏi một câu rồi đóng ứng dụng.

22. Ảnh, âm thanh và các lỗi dễ hiểu nhầm

Notebook gốc gửi ảnh đúng theo ý tưởng dùng trường `images`, nhưng phải có tệp `Collage_of_Nine_Dogs.jpg`. Lệnh tải ảnh lại đang được comment, nên môi trường mới không tự có ảnh. Khi chuyển sang thí nghiệm ảnh, chuẩn hóa cùng tệp, kích thước và câu hỏi cho mọi cấu hình.

Gemma có khả năng đa phương thức không có nghĩa mọi cách nhét dữ liệu vào API đều hợp lệ. Ô audio gốc đưa dữ liệu base64 vào `content`. Nó trở thành một chuỗi ký tự rất dài trong prompt, không phải bằng chứng model nhận được một waveform có ngữ nghĩa âm thanh.

Output lưu ở ô 23 cho thấy `prompt_eval_count=4095`, `eval_count=1`, `done_reason='length'` và content rỗng. Các dấu hiệu này **phù hợp với vấn đề giới hạn ngữ cảnh/đầu ra trong lời gọi đó**, nhưng chưa đủ để quy nguyên nhân duy nhất hay kết luận model không hỗ trợ audio.

Không đo "nhận dạng âm thanh" từ output này. Cần đối chiếu trường đầu vào âm thanh mà phiên bản API/backend hỗ trợ, định dạng media yêu cầu, sau đó làm smoke test có bản chép lời tham chiếu. Không mặc định mảng float32 mã hóa base64 sẽ được giải đúng chỉ vì model có khả năng audio.

Dấu hiệu	Kiểm tra trước
Model not found	Ô tải model, tên tag, đường dẫn model
Connection refused	Readiness và log server, đúng cổng 11435
Câu trả lời rỗng	done_reason, cap, think, backend và log
OOM	VRAM thực, context, draft, tác vụ GPU khác
Chậm bất thường	load_s, CPU offload, prompt dài, thinking
Không thấy thay đổi sau set env	Server đã được restart chưa?
Một tùy chọn không hợp lệ	Phiên bản API/engine và log

Phạm vi benchmark đi kèm là text để làm rõ các biến. Đa phương thức là một nhánh mở rộng cần bộ dữ liệu và phép đo riêng; không trộn ảnh và văn bản rồi báo một tốc độ token chung mà không giải thích khác biệt đầu vào.

23. Cách viết báo cáo và bảo vệ đồ án

Tên đề tài đề xuất: Thiết kế và đánh giá hệ thống chọn cấu hình suy luận Gemma 4 12B trên GPU T4 dưới ràng buộc bộ nhớ và chất lượng.

Phần đóng góp của nhóm gồm bộ quản lý thí nghiệm, giao thức đo có kiểm soát, bộ thu tài nguyên, bộ lọc cấu hình, predictor được train từ số đo và hệ thống demo. Engine, Gemma, các kernel attention và speculative có sẵn phải được ghi nguồn; không gọi đó là thuật toán nhóm tự phát minh.

Một báo cáo có thể đi theo các phần: bài toán và ràng buộc; nền tảng decoding; phân tích mã gốc; thiết kế hệ thống; dữ liệu và phương pháp; kết quả; phân tích lỗi/giới hạn; kết luận.

Câu hỏi phản biện	Câu trả lời cần có bằng chứng
Nhóm đã train gì?	Predictor độ trễ; đưa training_log và tệp model
Tại sao nhanh hơn?	Cặp đối chứng, log hỗ trợ, số token giống nhau
Chất lượng còn giữ không?	Điểm test, câu sai, rubric thủ công
Có chạy đúng T4 không?	hardware, driver, thời điểm và manifest
Predictor có hơn best-fixed không?	Bảng so trực tiếp, kể cả khi không hơn
Kết quả có tái hiện được không?	Code, seed, digest, phiên bản và dữ liệu thô

Mẫu câu kết luận có chỗ điền: “Trong [số] prompt test thuộc [nhóm], cùng model [digest], cấu hình [tên] đạt tỷ số thời gian baseline/optimized là [số], khoảng bootstrap [khoảng]. VRAM đỉnh lấy mẫu tối đa là [số]. Điểm tác vụ thay đổi từ [số] sang [số]. Kết quả giới hạn trong [phạm vi].”

Chỉ điền sau khi chạy. Nếu tốc độ không cải thiện, báo rằng cấu hình đó không tạo lợi ích trên phạm vi thử. Nếu predictor chọn bằng best-fixed, nói đúng điều đó. Một thí nghiệm có kiểm soát giải thích được kết quả âm vẫn có giá trị học thuật.

Muốn nâng mức nghiên cứu: bổ sung workload thực; thử chọn cấu hình bằng ít phép đo hơn exhaustive search; đo chi phí hiệu chỉnh và điểm hòa vốn; nghiên cứu tokenizer/context guard chính xác; thêm instrumentation acceptance của draft. Mỗi hướng cần đối chứng riêng.

24. Checklist và thuật ngữ để tự ôn

Trước khi chạy

- ☐ Chọn T4, kiểm tra dung lượng RAM/VRAM/đĩa.
- ☐ Dùng đúng model và cùng EXPERIMENT_ID.
- ☐ Biết QUICK chỉ để thử quy trình, không phải kết quả nghiệm thu đầy đủ.
- ☐ Giữ nguyên context, cap, sampling và thinking giữa các cấu hình.

Trước khi kết luận

- ☐ Đã chạy lại baseline trong cùng runtime với cấu hình tối ưu.
- ☐ Không có thay đổi digest hoặc engine bị bỏ qua.
- ☐ Có đủ các cặp đo, giữ cả lỗi và các trường hợp output bị cắt.
- ☐ Chỉ so bài cố định độ dài khi số token thực phù hợp.
- ☐ Phân biệt latency của người dùng, thời gian engine và thời gian nạp.
- ☐ Có đối chứng best-fixed, không chỉ baseline mặc định.
- ☐ Có đánh giá chất lượng ngoài độ dài và tốc độ.
- ☐ Chưa dùng test để sửa lựa chọn rồi tái gọi đó là test độc lập.

Từ	Nghĩa ngắn
Inference	Dùng model để tính đầu ra
Prefill	Xử lý các token đầu vào
Decode	Sinh các token mới
Context	Phạm vi token model có thể dùng trong lượt chạy
KV cache	Dữ liệu attention trung gian được giữ lại
Quantization	Biểu diễn số bằng độ chính xác thấp hơn
Draft / target	Thành phần đề xuất / model kiểm tra
Latency	Thời gian chờ của một công việc
Throughput	Lượng công việc hoàn thành trong một đơn vị thời gian
Ablation	Đối chiếu để tách tác dụng của một thành phần
OOM	Không đủ bộ nhớ cho thao tác đang thực hiện
Digest	Mã định danh theo nội dung artifact
Overfitting	Hợp quá mức với dữ liệu dùng để lựa chọn/học

Tự kiểm tra: vì sao nén cache 50% không giảm tổng VRAM 50%; vì sao phải giữ cùng chế độ thinking; vì sao predictor tốt chưa chắc tăng tốc?

25. Phạm vi đã làm và nguồn tham khảo

Đã có trong mã: ba notebook tự chứa; tải/xác minh model; khóa model, engine và dữ liệu; quản lý server; đo streaming và tài nguyên; lưu lỗi; tạo bộ kiểm thử; thử sáu cấu hình; train Random Forest; chọn cấu hình có ràng buộc; test cùng runtime; bootstrap; biểu đồ từ số đo thật và demo text.

Chưa được thực hiện trong môi trường bàn giao: chạy Gemma trên T4; tạo predictor đã train bằng số đo T4; chứng minh một mức tăng tốc cụ thể; fine-tune trọng số Gemma hoặc draft; đo acceptance nội bộ; bộ điều khiển speculative từng vòng; benchmark đa phương thức; đánh giá chất lượng tiếng Việt tổng quát.

Kiểm chứng phần mềm: xem VALIDATION.md trong ZIP để biết chính xác kiểm tra nào đã chạy. Kiểm thử CPU/mock kiểm tra tính toán và luồng dữ liệu; không thay cho kiểm thử CUDA và khả năng hỗ trợ option của engine trên T4.

Các nguồn dưới đây dùng để đối chiếu hành vi công cụ và nền tảng kỹ thuật. Số liệu, cấu hình lựa chọn và ví dụ giảng giải trong tài liệu là thiết kế riêng của gói, trừ khi ghi rõ là thông tin nguồn hoặc output của notebook gốc.

- **S1. Ollama - gemma4:12b.** Artifact, tham số và các thành phần phân phối: https://ollama.com/library/gemma4:12b
- **S2. Ollama - FAQ.** Cấu hình server, cache, GPU và vòng đời model: https://docs.ollama.com/faq
- **S3. Ollama - Generate a chat message.** Trường API và đơn vị thống kê: https://docs.ollama.com/api/chat
- **S4. Ollama - Modelfile reference.** Tham số generation và draft_num_predict: https://docs.ollama.com/modelfile
- **S5. Ollama - Thinking.** Cách yêu cầu chế độ thinking: https://docs.ollama.com/capabilities/thinking
- **S6. Leviathan, Kalman, Matias.** Fast Inference from Transformers via Speculative Decoding, 2023: https://arxiv.org/abs/2211.17192
- **S7. Dao và cộng sự.** FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, 2022: https://arxiv.org/abs/2205.14135
- **S8. NVIDIA.** GPU Performance Background User's Guide: https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html
- **S9. scikit-learn.** RandomForestRegressor: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html
- **S10. Google Colab - FAQ.** Tài nguyên và giới hạn runtime: https://research.google.com/colaboratory/faq.html

Ngày đối chiếu nguồn: 13/10/2026. Với lỗi phiên bản cụ thể, ưu tiên metadata và log của phiên chạy thực tế hơn các giá trị mặc định mô tả trong một trang tài liệu.